

dj



Formulaire

Les formulaires DJANGO

- Rôle de Django
 - Django intervient dans la gestion des formulaires à trois niveaux:
 - Réparation et restructuration des données en vue de leur présentation
 - Création des formulaires HTML pour les données
 - Réception et traitement des formulaires et des données envoyés par le client
 - Django offre un service de formulaire autour des classes `django.forms.Form`, `django.forms.ModelForm`, et les classes de gestion de champs `django.forms.xxxFields`

Les formulaires DJANGO

- La classe Form : présentation
- On peut définir un formulaire en écrivant une classe héritant de la classe **Form**
- Les attributs de cette classe sont initialisés avec des instances de classes de champs
- L'objet instancié de cette classe est directement injectable dans un template

```
# dans forms.py  
from django import forms
```

```
class NameForm(forms.Form):  
    your_name = forms.CharField(label='Your name', max_length=100)
```

```
# dans views.py  
from .forms import NameForm
```

```
def get_form() :  
    form = NameForm()  
    return render(request, 'form.html', {'form': form})
```

Les formulaires DJANGO

- La classe Form : affichage

```
# dans forms.html
<form action="/your-name/" method="post">
  {% csrf_token %}
  {{ form }}
  <input type="submit" value="Submit">
</form>
```

```
# page html générée
```

```
...
<label for="your_name">Your name: </label>
<input id="your_name" type="text" name="your_name" maxlength="100" required>
...
```

Les formulaires DJANGO

- La classe Form : soumission
 - La vue qui traite la soumission du formulaire doit mettre en rapport l'objet formulaire avec les données POST de la requête
 - La classe Form fournit une méthode `is_valid()` qui procède aux routines de validation pour tous ses champs.
 - Lorsque la méthode est appelée et que tous les champs contiennent des données valables, celle-ci insère les données du formulaire dans l'attribut `cleaned_data` et renvoie la valeur True.
 - L'attribut `is_bound` permet de savoir si le formulaire est lié

```
# dans views.py
...
def get_form() :
    if request.method == 'POST':
        form = NameForm(request.POST) # formulaire « lié »
        if form.is_valid():
            # gérer les données dans form.cleaned_data
            return HttpResponseRedirect('/thanks/')
    else :
        form = NameForm() # formulaire « non lié »
        return render(request, 'form.html', {'form': form})
```

Les formulaires DJANGO

- La classe Form : widgets de champs
- Les classes Fields permettent d'implémenter les besoins de validation les plus courants.
- Chaque classe xxxField correspond à une classe xxxInput par l'attribut **widget**
- Chaque champs possède une valeur par défaut
- Chaque champ peut être normalisé dans le type de donnée qu'il gère
- Chaque champ est considéré obligatoire (required=True) par défaut
- On accède aux champs d'une instance de Form depuis son attribut **form.fields**

```
class ContactForm(forms.Form):
    subject = forms.CharField(max_length=100) # input
    message = forms.CharField(widget=forms.Textarea) # textarea
    sender = forms.EmailField()
    cc_myself = forms.BooleanField(required=False) # checkbox

f = ContactForm()
f.fields['subject']
for row in f.fields.values():
    print(row)
```

<https://docs.djangoproject.com/fr/3.1/ref/forms/fields/>

Les formulaires DJANGO

- La classe Form : traitement des données postées

```
from django.core.mail import send_mail

if form.is_valid():
    subject = form.cleaned_data['subject']
    message = form.cleaned_data['message']
    sender = form.cleaned_data['sender']
    cc_myself = form.cleaned_data['cc_myself']

    recipients = ['info@example.com']
    if cc_myself:
        recipients.append(sender)

    send_mail(subject, message, sender, recipients)
    return HttpResponseRedirect('/thanks/')
```

Les formulaires DJANGO

- Stratégies d'affichage : en bloc
- Avantages : simple, rapide
- Défaut : dépendant totalement de Django

dans form.html

```
{{ form.as_table }} # affichage sous forme de cellules <td> à l'intérieur de lignes <tr>  
{{ form.as_p }} # affichage dans des balises <p>  
{{ form.as_ul }} affichage dans des balises <ul>, <li>
```


Les formulaires DJANGO

- Stratégies d'affichage : personnalisable
- Avantages : flexible
- Défaut : plus lent à écrire pour intégrer les erreurs, les labels et les champs

```
# dans form.html
{{ form.non_field_errors }}
<div class="fieldWrapper">
    {{ form.subject.errors }}
    <label for="{{ form.subject.id_for_label }}">Email subject:</label>
    {{ form.subject }}
</div>
<div class="fieldWrapper">
    {{ form.message.errors }}
    <label for="{{ form.message.id_for_label }}">Your message:</label>
    {{ form.message }}
</div>
...
```

Les formulaires DJANGO

- Stratégies d'affichage : par itération
- Avantages : réutilisable via `{% include %}`, rapide, légèrement personnalisable
- Défaut : peu flexible, exige un traitement particulier des champs « hidden » avec `form.hidden_fields` et `form.visible_fields`

```
# dans generic_form.html
{% for field in form.visible_fields %}
  <div class="fieldWrapper">
    {{ field.errors }}
    {{ field.label_tag }} {{ field }}
    {% if field.help_text %}
    <p class="help">{{ field.help_text|safe }}</p>
    {% endif %}
  </div>
{% endfor %}
```

```
# dans form.html
{% include "generic_form.html" with form=contact_form %}
```

Les formulaires DJANGO

- Valeurs initiales dynamiques
- On peut placer des valeurs dans les champs à l'initialisation du formulaire de 2 façons
- Par le formulaire avec le paramètre nommé **initial** dans le constructeur du formulaire
- Par le champ avec le paramètre nommé **initial** dans le constructeur du champ. Celui ci écrase le premier si les deux sont définis

```
# par le formulaire
f = ContactForm(initial={'subject': 'Hi there!'})
...

# par les champs
class ContactForm(forms.Form):
    subject = forms.CharField(initial='Hi there !')

f = ContactForm()
```

Les formulaires DJANGO

- Valeurs modifiées
- On peut comparer des valeurs initiales aux valeurs retournées après soumission du formulaire
- La comparaison peut s'exprimer comme booléen avec `form.has_changed()`
- Les données modifiées sont accessibles dans `form.changed_data`

```
f = ContactForm(request.POST, initial=data) # comparaison
if f.has_changed():
    print("Paramètres modifiés: {}".format(", ".join(f.changed_data)))
```

Les formulaires DJANGO

- La validation : principe
 - La validation s'opère au travers une procédure qu'on appelle le **nettoyage des données**
 - Ce nettoyage est déclenché par l'appel à la méthode **form.is_valid()**
 - Il existe un nettoyage au niveau de la méthode **clean** de chaque champ :
 - appel à la méthode **to_python()** de conversion de la valeur insérée dans le type exigé
 - appel à la méthode **validate()** pour validation spécifique => à surcharger
 - appel à la méthode **run_validators()** d'application de validateurs génériques
 - ces méthodes sont peuvent renvoyer des exception **ValidationError**
 - Il existe également un nettoyage spécifique postérieur au niveau du formulaire par appels successifs à des méthodes **clean_nom_du_champ**, agissant sur les champs de **cleaned_data**
 - On peut enfin surcharger la méthode **clean()** au niveau du formulaire, quand on veut baser une validation sur des interactions entre champs

Les formulaires DJANGO

- La validation : Générer des erreurs
- ValidationError s'instancie avec un **texte**, un **code**, et des paramètres optionnels **params**

```
raise ValidationError(  
    _('Invalid value: %(value)s'),  
    code='invalid',  
    params={'value': '42'},  
)
```

- On peut lever plusieurs exceptions ValidationError d'un coup

```
raise ValidationError([  
    ValidationError(_('Error 1'), code='error1'),  
    ValidationError(_('Error 2'), code='error2'),  
])
```


Les formulaires DJANGO

- La validation : en Pratique (nettoyage générique d'un champ)

```
from django import forms
from django.core.validators import validate_email

class MultiEmailField(forms.Field):

    default_validators = [validate_email]

    def to_python(self, value):
        """Normalize data to a list of strings."""
        # Return an empty list if no input was given.
        if not value:
            return []
        return value.split(',')

    def validate(self, value):
        """Check if value consists only of valid emails."""
        # Use the parent's handling of required fields, etc.
        super().validate(value)
```

Les formulaires DJANGO

- La validation : en Pratique (nettoyage spécifique d'un champ)

```
from django import forms
from django.core.exceptions import ValidationError

class ContactForm(forms.Form):
    ...
    def clean_recipients(self):
        data = self.cleaned_data['recipients']
        if "fred@example.com" not in data:
            raise ValidationError("You have forgotten about Fred!", code='forgotten')

        # la valeur retournée écrasera la valeur dans cleaned_data
        return data
```

Les formulaires DJANGO

- La validation : en Pratique (nettoyage sur plusieurs champ)

```
from django import forms
```

```
class ContactForm(forms.Form):
```

```
    def clean(self):
```

```
        cleaned_data = super().clean()
```

```
        cc_myself = cleaned_data.get("cc_myself")
```

```
        subject = cleaned_data.get("subject")
```

```
        if cc_myself and subject and "help" not in subject:
```

```
            msg = "Must put 'help' in subject when cc'ing yourself."
```

```
            self.add_error('cc_myself', msg)
```

```
            self.add_error('subject', msg)
```

add_error() enlève automatiquement le champ de cleaned_data.

add_error() accepte en second paramètre une chaîne de caractère ou une instance de ValidationError

Les formulaires DJANGO

- Upload de fichiers
 - On utilise le composant `FileField`
 - Les fichiers sont placés dans `request.FILES`
 - Les fichiers uploadés sont encapsulés dans un objet `UploadedFile`
 - L'appel à `UploadedFile.chunks()` permet de déplacer ces fichiers de façon souple

```
from .forms import UploadFileForm
```

```
def upload_file(request):
```

```
    if request.method == 'POST':
```

```
        form = UploadFileForm(request.POST, request.FILES)
```

```
        if form.is_valid():
```

```
            f = request.FILES[0]
```

```
            with open('some/file/name.txt', 'wb+') as destination:
```

```
                for chunk in f.chunks():
```

```
                    destination.write(chunk)
```

```
# upload multiple
```

```
multi_upload = forms.FileField(widget=forms.ClearableFileInput(attrs={'multiple': True}))
```

Les formulaires DJANGO

- **Vues génériques d'édition : FormView**
- Une vue qui affiche un formulaire.
- En cas d'erreur, le formulaire est ré-affiché avec les erreurs de validation rencontrées
- En cas de succès, l'utilisateur est redirigé vers une nouvelle URL

```
from myapp.forms import ContactForm
from django.views.generic.edit import FormView
```

```
class ContactView(FormView):
    template_name = 'contact.html'
    form_class = ContactForm
    success_url = '/thanks/'
```

```
def form_valid(self, form):
    # surcharger les traitement post validation.
    ...
    return super().form_valid(form)
```

Les formulaires DJANGO

- **Vues génériques d'édition : CreateView**
 - Une vue qui affiche un formulaire permettant la création d'objet
 - Ré-affiche le formulaire en cas d'erreurs de validation.
 - En cas de succès, l'objet est enregistré.
 - On peut suffixer le template par défaut avec l'attribut `template_name_suffix`
 - On peut sélectionner les champs du modèle éligibles à l'édition avec l'attribut `fields`
 - On a accès à l'objet en cours de création dans la vue avec `self.object`

```
from django.views.generic.edit import CreateView
from myapp.models import Author
```

```
class AuthorCreate(CreateView):
    model = Author
    template_name_suffix = '_create'
    fields = ['name']
```

Même structure pour UpdateView et DeleteView